TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

MÔN HỌC

KĨ THUẬT LẬP TRÌNH NÂNG CAO

ĐỀ TÀI: Mô phỏng bài toán xếp lịch thi tối ưu

GIÁO VIÊN HƯỚNG DẪN : THS.NGHIÊM VĂN TÍNH
VIÊN THỰC HIỆN : NGUYỄN TIẾN THẮNG
LỚP : 58KTP

THÁI NGUYÊN - 2026

TRƯỜNG ĐHKTCN	**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM**
KHOA ĐIỆN TỬ	***Độc lập - Tự do - Hạnh phúc***

NHIỆM VỤ BÀI TẬP LỚN MÔN KĨ THUẬT LẬP TRÌNH NÂNG CAO

Sinh viên : Nguyễn Tiến Thắng MSV : K225480106058

Lớp: K58KTP **Ngành:** Kỹ thuật máy tính

Bộ môn: Công Nghệ Thông Tin

Giáo viên hướng dẫn: ThS. Nghiêm Văn Tính

1. Tên đề tài : Mô phỏng bài toán xếp lich thi tối chiếu
2. Nội dung các phần thuyết minh và tính toán
- Trình bày cơ sở lý thuyết liên quan.
- Phân tích bài toán xếp lịch thi.
-Trình bày thuật toán sử dụng.
-Đánh giá độ phức tạp của thuật toán
3. Các sản phẩm, kết quả :
- Xây dựng chương trình hoàn chỉnh bằng một ngôn ngữ cụ thể *(C/C++, Java hoặc Python).*
- Trình bày và phân tích kết quả thu được

4. Ngày giao nhiệm vụ : / /2026

5. Ngày hoàn thành nhiệm vụ : 15 / 03 /2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

TRƯỜNG ĐHKTCN	**CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM**
KHOA ĐIỆN TỬ	***Độc lập – Tự do – Hạnh phúc***

PHIẾU GHI ĐIỂM

HƯỚNG DẪN BÀI TẬP LỚN MÔN KĨ THUẬT LẬP TRÌNH NÂNG CAO

Sinh viên : Nguyễn Tiến Thắng MSV : K225480106058

Lớp: K58KTP **Ngành:** Kỹ thuật máy tính

Giáo viên hướng dẫn: ThS. Nghiêm Văn Tính

Đề tài: Mô phỏng bài toán xếp lịch thi tối ưu

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

……

……

……

……

……

……

Xếp loại : …… Điểm : ……

Thái Nguyên, ngày tháng năm 2025

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc đến ThS. Nguyêm Văn Tính, người đã tận tình hướng dẫn và đồng hành cùng em trong suốt quá trình thực hiện đề tài. Thầy luôn dành thời gian chỉ bảo, định hướng và giúp em hiểu rõ hơn về nội dung nghiên cứu cũng như phương pháp triển khai một cách khoa học và hiệu quả.

Trong quá trình thực hiện, em đã nhận được rất nhiều góp ý quý báu và sự hỗ trợ nhiệt tình từ Thầy. Những nhận xét chuyên môn sâu sắc cùng sự tận tâm của Thầy không chỉ giúp em hoàn thiện đề tài mà còn giúp em nâng cao kiến thức, rèn luyện tư duy nghiên cứu và kỹ năng làm việc nghiêm túc.

Em xin chân thành cảm ơn Thầy vì sự hướng dẫn, động viên và những kiến thức quý báu mà Thầy đã truyền đạt. Sự quan tâm và giúp đỡ của Thầy là nguồn động lực lớn để em hoàn thành đề tài này. Em xin trân trọng cảm ơn!

Mục Lục

LỜI CẢM ƠN 3
DANH MỤC HÌNH ẢNH 6
LỜI NÓI ĐẦU 7
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI VÀ KHẢO SÁT 8
1.1 Tổng quan về bài toán xếp lịch thi 8
1.1.1 Khái niệm và đặc điểm bài toán 8
1.1.2 Ý nghĩa thực tiễn của bài toán 8
1.2 Thực trạng các phương pháp xếp lịch thi 9
1.2.1 Các phương pháp truyền thống 9
1.2.2 Các phương pháp tối ưu hiện đại 9
1.3 Mục tiêu và phạm vi nghiên cứu 10
1.3.1 Mục tiêu đề tài 10
1.3.2 Phạm vi thực hiện 10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH BÀI TOÁN 11
2.1 Mô hình hóa bài toán xếp lịch thi 11
2.1.1 Các thành phần của bài toán 11
2.1.2 Các ràng buộc cứng và ràng buộc mềm 12
2.2 Biểu diễn bài toán dưới dạng toán học 14
2.2.1 Biểu diễn bằng đồ thị xung đột 14
2.2.2 Hàm mục tiêu tối ưu 14
2.3 Các thuật toán giải bài toán 15
2.3.1 Thuật toán tham lam 15
2.3.2 Thuật toán heuristic nâng cao 16
2.3.3 Đánh giá độ phức tạp thuật toán 16
CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT CHƯƠNG TRÌNH 18
3.1 Môi trường và công cụ lập trình 18
3.1.1 Ngôn ngữ lập trình và thư viện 18
3.1.2 Cơ sở dữ liệu và lưu trữ 18
3.2 Thiết kế hệ thống 19
3.2.1 Kiến trúc chương trình 19
3.2.2 Thiết kế dữ liệu và bảng cơ sở dữ liệu 20

3.3 Cài đặt thuật toán xếp lịch 22
3.3.1 Xử lý dữ liệu đầu vào 22
3.3.2 Thực hiện thuật toán xếp lịch 22
3.3.3 Kiểm tra ràng buộc 23
3.4 Thiết kế giao diện người dùng 24
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 26
4.1 Dữ liệu và phương pháp thực nghiệm 26
4.2 Thực hiện thực nghiệm và thu thập kết quả 27
4.2 Kết quả chạy chương trình 28
4.2.1 Kết quả xếp lịch và so sánh các cấu hình 28
4.2.2 Kiểm tra ràng buộc 30
4.3 Đánh giá hiệu quả thuật toán 31
4.4 So sánh và nhận xét 31
CHƯƠNG 5. NHẬN XÉT VÀ KẾT LUẬN 33
5.1 Kết quả đạt được 33
5.2 Một số điểm cần hoàn thiện thêm 33
5.3 Hướng phát triển trong tương lai 34
TÀI LIỆU THAM KHẢO 35

DANH MỤC HÌNH ẢNH

Bảng 2.1. Tổng hợp ràng buộc của bài toán (đơn vị: ngày, ca, chỗ). 12
Hình 3.2. Sơ đồ quy trình xếp lịch thi trong hệ thống 20
Hình 3.2 Biểu đồ Diagrams.....21
Hình 3.3 Giao diện sử dụng tính năng tối ưu.....22
Hình 3.4 Giao diện người dùng.....24
Hình 4.1 Chức năng Tạo dữ liệu mẫu 27
Bảng 4.2. Kết quả thực nghiệm theo cấu hình 28
Hình 4.3. So sánh số ngày thi, số ca và thời gian chạy 29
Hình 4.4 Kết quả chạy giảm áp lực ôn tập, tránh thi dồn. 29
Hình 4.5 Kết quả tiết kiệm chi phí tổ chức, thời gian kỳ thi ngắn. 30
Hình 4.6 kết quả cân bằng : tối ưu chi phí và giảm áp lực thi 30

LỜI NÓI ĐẦU

Trong bối cảnh số lượng sinh viên ngày càng tăng và quy mô tổ chức thi ngày càng phức tạp, việc xây dựng một thời khóa biểu thi hợp lý, khoa học và tối ưu trở thành một yêu cầu quan trọng đối với các cơ sở giáo dục. Bài toán xếp lịch thi không chỉ cần đảm bảo các ràng buộc bắt buộc như không trùng lịch đối với sinh viên, phù hợp với sức chứa phòng thi, mà còn hướng đến việc tối ưu hóa nguồn lực và giảm thiểu xung đột trong quá trình tổ chức. Đây là một bài toán thực tiễn có tính ứng dụng cao, đồng thời cũng là một vấn đề tối ưu tổ hợp điển hình trong lĩnh vực khoa học máy tính.

Xuất phát từ nhu cầu thực tế đó, đề tài "Mô phỏng bài toán xếp lịch thi tối ưu" được thực hiện nhằm nghiên cứu cơ sở lý thuyết, xây dựng mô hình và áp dụng các thuật toán phù hợp để giải quyết bài toán xếp lịch thi. Bên cạnh việc tìm hiểu các ràng buộc và tiêu chí tối ưu, đề tài còn tập trung xây dựng chương trình mô phỏng, thực nghiệm với dữ liệu mẫu và đánh giá hiệu quả của phương pháp đề xuất.

Thông qua đề tài này, người thực hiện mong muốn góp phần minh họa cách tiếp cận bài toán xếp lịch thi bằng phương pháp tin học, đồng thời rèn luyện khả năng phân tích bài toán, thiết kế thuật toán và triển khai chương trình ứng dụng trong thực tế. Đây cũng là cơ hội để vận dụng kiến thức đã học vào giải quyết một vấn đề cụ thể mang tính thực tiễn cao.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI VÀ KHẢO SÁT

1.1 Tổng quan về bài toán xếp lịch thi

1.1.1 Khái niệm và đặc điểm bài toán

Bài toán xếp lịch thi (Exam Timetabling) là bài toán gán mỗi môn thi vào một ca thi cụ thể và phân bổ thí sinh vào các phòng thi sao cho thỏa mãn các ràng buộc đã đặt ra. Dữ liệu đầu vào thường bao gồm: danh sách môn thi, danh sách đăng ký của sinh viên, các ca thi theo ngày và giờ, danh sách phòng thi cùng sức chứa, và các quy định nghiệp vụ liên quan.

Đặc trưng quan trọng của bài toán gồm: (i) số lượng tổ hợp gán lịch rất lớn; (ii) tồn tại nhiều ràng buộc cứng bắt buộc phải thỏa mãn, như xung đột sinh viên hoặc phòng không thể sử dụng đồng thời; (iii) ngoài ràng buộc cứng còn có các ràng buộc mềm nhằm nâng cao chất lượng lịch thi, chẳng hạn giãn cách ngày thi, giảm số ngày thi hoặc ưu tiên ca thi sớm. Do đó, bài toán thường được xem là một bài toán tối ưu tổ hợp, khó đạt nghiệm tối ưu tuyệt đối khi quy mô dữ liệu lớn, và trong thực tế thường áp dụng các thuật toán heuristic hoặc greedy.

Xét theo quy mô, độ phức tạp của bài toán tăng nhanh theo ba chiều: số môn thi, số ca thi và số lượng sinh viên. Khi số lượng đăng ký lớn, mỗi quyết định gán ca cho một môn sẽ ảnh hưởng đến nhiều sinh viên, làm thay đổi không gian nghiệm khả thi của các môn còn lại. Vì vậy, khi thiết kế thuật toán thường cần ưu tiên:

- Thỏa mãn ràng buộc cứng trước tiên.
- Tạo lời giải nhanh để có thể chạy lại nhiều lần khi cần.
- Có khả năng giải thích và kiểm chứng kết quả, do lịch thi là dữ liệu nghiệp vụ cần công khai.

Một điểm đặc thù của bài toán xếp lịch thi so với xếp thời khóa biểu là mỗi môn thi thường chỉ diễn ra một lần trong kỳ nhưng số lượng thí sinh rất lớn và cần phân phòng chi tiết. Vì vậy, vấn đề sức chứa phòng và phân bổ sinh viên vào phòng trở thành thành phần không thể tách rời khỏi quá trình gán ca thi.

1.1.2 Ý nghĩa thực tiễn của bài toán

Việc xếp lịch thi thủ công dễ phát sinh sai sót như trùng ca của sinh viên, vượt sức chứa phòng hoặc sử dụng phòng không khả dụng. Ngoài ra, khi quy mô lớn, quá trình lập lịch thủ công cũng tốn nhiều thời gian và khó điều chỉnh khi có thay đổi.

Một công cụ hỗ trợ xếp lịch thi tự động mang lại nhiều lợi ích:

- Giảm công sức cho cán bộ khảo thí khi lập lịch cho nhiều môn và nhiều lớp.
- Hạn chế sai sót nhờ kiểm tra ràng buộc tự động.
- Tạo lịch thi có chất lượng tốt hơn theo các tiêu chí tối ưu như giãn cách hợp lý hoặc giảm số ngày thi.
- Dễ dàng xuất lịch sang định dạng Excel để thông báo, in ấn và lưu trữ.

1.2 Thực trạng các phương pháp xếp lịch thi

1.2.1 Các phương pháp truyền thống

Trong nhiều đơn vị, lịch thi được xây dựng theo phương pháp thủ công: tổng hợp danh sách môn, lựa chọn ngày và ca thi cho từng môn, sau đó phân phòng theo sức chứa và kiểm tra xung đột sinh viên bằng bảng tính. Phương pháp này phù hợp với quy mô nhỏ nhưng gặp nhiều hạn chế khi dữ liệu tăng, như khó kiểm tra đầy đủ xung đột, khó tối ưu theo nhiều tiêu chí và khó cập nhật khi có thay đổi phát sinh.

1.2.2 Các phương pháp tối ưu hiện đại

Các phương pháp hiện đại thường mô hình hóa bài toán dưới dạng bài toán tối ưu và áp dụng:

- Thuật toán heuristic hoặc greedy để tìm lời giải nhanh.
- Meta-heuristic như tìm kiếm cục bộ, mô phỏng tôi luyện (simulated annealing), thuật toán di truyền…
- Phương pháp tối ưu chính xác như ILP hoặc CP-SAT khi quy mô vừa và ràng buộc rõ ràng.

Trong phạm vi hệ thống này, giải pháp greedy có chấm điểm (scoring) được ưu tiên nhằm cân bằng giữa tốc độ xử lý và khả năng đáp ứng các ràng buộc thực tế. Thuật toán được thiết kế để hoạt động hiệu quả khi số lượng đăng ký lớn và dễ mở rộng thêm các ràng buộc mềm khi cần.

Ngoài yếu tố thuật toán, khả năng vận hành thực tế cũng rất quan trọng. Dữ liệu có thể thay đổi sát ngày thi như thêm hoặc hủy đăng ký, khóa phòng đột xuất. Vì vậy hệ thống cần chạy lại nhanh, ghi log để truy vết và đảm bảo không làm ảnh hưởng lịch đã lưu. Điều này được thực hiện thông qua cơ chế tối ưu trong transaction và rollback khi xuất Excel.

1.3 Mục tiêu và phạm vi nghiên cứu

1.3.1 Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng chương trình hỗ trợ xếp lịch thi tự động với các yêu cầu chính:

- Tạo lịch thi từ dữ liệu đăng ký môn của sinh viên.
- Đảm bảo ràng buộc cứng: không trùng ca của sinh viên, phòng không trùng ca, phòng sẵn sàng và đủ sức chứa.
- Hỗ trợ tiêu chí tối ưu ở mức heuristic qua các chế độ: giãn cách (spread), gom lịch (compact) và cân bằng (balanced).
- Cung cấp giao diện web để chạy tối ưu, xem lịch và xuất Excel phục vụ nghiệp vụ.

1.3.2 Phạm vi thực hiện

Phạm vi thực hiện tập trung vào các đối tượng và nghiệp vụ chính gồm: môn thi, sinh viên, đăng ký, ca thi, phòng thi, lịch thi và phân công giám thị.

Các nội dung ngoài phạm vi nghiên cứu bao gồm: tối ưu phân công giám thị theo quy tắc phức tạp, ràng buộc theo lớp hoặc khoa, ưu tiên phòng đặc biệt theo hình thức thi hoặc đồng bộ lịch với các hệ thống đào tạo khác.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH BÀI TOÁN

2.1 Mô hình hóa bài toán xếp lịch thi

2.1.1 Các thành phần của bài toán

Trong hệ thống xếp lịch thi, dữ liệu được tổ chức thành các thành phần cơ bản nhằm mô hình hóa đầy đủ các yếu tố liên quan đến việc lập lịch. Các thành phần chính bao gồm:

- Tập môn thi M: được lấy từ bảng *MonThi*, bao gồm các thông tin như mã môn, tên môn, thời lượng và hình thức thi.
- Tập sinh viên S: được lấy từ bảng *SinhVien*, gồm mã sinh viên, lớp và khoa.
- Quan hệ đăng ký thi $E \subseteq S \times M$: được lấy từ bảng *DangKy*, biểu diễn mối quan hệ giữa sinh viên và các môn mà sinh viên đăng ký thi.
- Tập ca thi T: được lấy từ bảng *CaThi*, mô tả các khung thời gian tổ chức thi, bao gồm ngày thi, thứ tự ca trong ngày và thời gian bắt đầu – kết thúc.
- Tập phòng thi R: được lấy từ bảng *PhongThi*, gồm mã phòng và sức chứa của phòng (đơn vị: số chỗ ngồi).
- Ma trận khả dụng $A \subseteq T \times R$: được lấy từ bảng *CaThi_PhongThi*, thể hiện trạng thái sử dụng phòng trong từng ca thi (sẵn sàng hoặc bị khóa).

Đầu ra của bài toán là lịch thi L, được lưu trữ trong bảng *LichThi*. Mỗi dòng của bảng này biểu diễn việc một môn thi được tổ chức tại một phòng cụ thể trong một ca cụ thể. Bên cạnh đó, bảng *LichThi_SinhVien* lưu thông tin phân bổ sinh viên vào từng phòng thi tương ứng với lịch thi đã tạo.

Mô hình dữ liệu được thiết kế sát với nghiệp vụ thực tế và hỗ trợ thuận lợi cho việc triển khai thuật toán. Hai đặc điểm nổi bật của mô hình gồm:

- Ca thi được xác định rõ ràng theo ngày, thứ tự trong ngày và thời gian bắt đầu – kết thúc, giúp hiển thị trực quan trên giao diện và đảm bảo thứ tự duyệt ca ổn định trong quá trình xếp lịch.
- Trạng thái khả dụng của phòng theo từng ca thi, được lưu trong bảng *CaThi_PhongThi*, cho phép mô tả linh hoạt các tình huống thực tế như phòng chỉ hoạt động trong một số ca hoặc bị khóa đột xuất. Thuật toán chỉ cần lọc các phòng có trạng thái *SanSang* để đảm bảo ràng buộc này.

2.1.2 Các ràng buộc cứng và ràng buộc mềm

Trong bài toán xếp lịch thi, các điều kiện cần thỏa mãn được chia thành hai nhóm: ràng buộc cứng (hard constraints) và ràng buộc mềm (soft constraints).

Ràng buộc cứng là các điều kiện bắt buộc phải thỏa mãn. Nếu bất kỳ ràng buộc cứng nào bị vi phạm, lịch thi được xem là không hợp lệ. Trong thực tế, các ràng buộc này thường xuất phát từ quy chế tổ chức thi, chẳng hạn:

- Một sinh viên không được thi hai môn trong cùng một thời điểm.
- Một phòng thi không được sử dụng cho nhiều môn trong cùng một ca.
- Số sinh viên được xếp vào phòng không vượt quá sức chứa của phòng.
- Chỉ sử dụng phòng thi có trạng thái sẵn sàng trong ca tương ứng.
- Một Phòng thi đầy đủ 2 giáo viên coi thi, giáo viên chỉ được coi 1 ca trong cùng 1 phòng thi

Ngược lại, ràng buộc mềm là các tiêu chí giúp đánh giá mức độ "tốt" của lịch thi. Vi phạm ràng buộc mềm không làm lịch thi vô hiệu, nhưng ảnh hưởng đến chất lượng tổng thể của phương án xếp lịch. Các ràng buộc mềm thường xuất phát từ mục tiêu tối ưu vận hành hoặc nâng cao trải nghiệm cho sinh viên, ví dụ:

- Giảm số ngày thi tổng thể để tiết kiệm chi phí tổ chức.
- Giãn cách thời gian thi giữa các môn của cùng một sinh viên để có thời gian ôn tập.
- Phân bố đều số lượng sinh viên giữa các phòng thi để giảm quá tải.

Việc phân loại ràng buộc thành hai nhóm giúp thiết kế thuật toán xếp lịch một cách rõ ràng và hiệu quả. Các phương án vi phạm ràng buộc cứng sẽ bị loại bỏ ngay từ đầu. Trong số các phương án còn lại, thuật toán sẽ đánh giá và lựa chọn phương án tối ưu dựa trên mức độ thỏa mãn các ràng buộc mềm thông qua cơ chế chấm điểm hoặc hàm mục tiêu

Bảng 2.1. Tổng hợp ràng buộc của bài toán (đơn vị: ngày, ca, chỗ).

Nhóm	Ràng buộc	Mô tả ngắn	Nơi kiểm soát
Cứng	Không trùng phòng theo ca	Mỗi phòng chỉ được sử dụng tối đa một lần trong cùng một ca thi	Ràng buộc UNIQUE (MaPhong, MaCa) trong CSDL

Nhóm	Ràng buộc	Mô tả ngắn	Nơi kiểm soát
Cứng	Không trùng ca của sinh viên	Một sinh viên không được thi hai môn trong cùng một ca	Thuật toán kiểm tra trước khi gán lịch
Cứng	Phòng phải sẵn sàng	Chỉ sử dụng phòng có trạng thái *SanSang* tại ca tương ứng	Lọc theo bảng CaThi_PhongThi
Cứng	Đủ sức chứa	Tổng sức chứa các phòng được phân phải lớn hơn hoặc bằng số sinh viên đăng ký	Thuật toán kiểm tra tổng sức chứa
Mềm	Giãn cách ngày thi	Ưu tiên tăng khoảng cách giữa các ngày thi của sinh viên	Chế độ *spread* / *balanced* (chấm điểm)
Mềm	Gom lịch	Ưu tiên tổ chức thi trong ít ngày và các ca sớm	Chế độ *compact* / *balanced* (chấm điểm)
Mềm	Khoảng cách tối thiểu	Nếu min_gap_days > 0, các môn phải cách nhau tối thiểu X ngày	Kiểm tra theo lịch sử thi của sinh viên

Bảng trên cho thấy hệ thống áp dụng đồng thời hai cơ chế đảm bảo tính hợp lệ của lịch thi: ràng buộc ở mức cơ sở dữ liệu và ràng buộc ở mức logic thuật toán.

Các ràng buộc được triển khai tại cơ sở dữ liệu thông qua các cơ chế như UNIQUE, CHECK và khóa ngoại (FOREIGN KEY) giúp ngăn chặn ngay từ đầu các trạng thái dữ liệu không hợp lệ, ví dụ như việc trùng phòng trong cùng một ca thi hoặc tham chiếu đến dữ liệu không tồn tại. Nhờ đó, tính toàn vẹn dữ liệu được đảm bảo ngay tại tầng lưu trữ.

Trong khi đó, các ràng buộc phụ thuộc vào trạng thái động của hệ thống — chẳng hạn như xung đột lịch thi của sinh viên, khả năng phân bổ theo sức chứa hoặc khoảng cách giữa các lần thi — được kiểm soát bởi logic của thuật toán xếp lịch. Những ràng buộc này cần được đánh giá liên tục dựa trên dữ liệu đăng ký và lịch sử phân bổ trong quá trình xây dựng lịch.

Việc kết hợp hai lớp kiểm soát giúp giảm thiểu rủi ro sai lệch dữ liệu và nâng cao độ tin cậy của hệ thống. Cơ sở dữ liệu đóng vai trò kiểm soát tính nhất quán cấu trúc, trong khi thuật toán xử lý các ràng buộc phụ thuộc ngữ cảnh và mục tiêu tối ưu hóa

2.2 Biểu diễn bài toán dưới dạng toán học

2.2.1 Biểu diễn bằng đồ thị xung đột

Một cách biểu diễn phổ biến của bài toán xếp lịch thi là sử dụng đồ thị xung đột giữa các môn thi. Trong đồ thị này:

- Mỗi đỉnh biểu diễn một môn thi.
- Một cạnh được nối giữa hai đỉnh nếu tồn tại ít nhất một sinh viên đăng ký cả hai môn tương ứng.

Khi đó, yêu cầu "một sinh viên không được thi hai môn trong cùng một ca" tương đương với việc hai đỉnh có cạnh nối không được gán cùng một màu, trong đó mỗi màu đại diện cho một ca thi. Bài toán xếp lịch thi vì vậy có thể được xem như một bài toán tô màu đồ thị với số màu tương ứng với số ca thi khả dụng.

Trong hệ thống hiện tại, thay vì xây dựng đồ thị xung đột một cách tường minh, thuật toán kiểm tra xung đột trực tiếp trên dữ liệu sinh viên. Cụ thể, với mỗi sinh viên, hệ thống lưu tập các ca đã được gán trong cấu trúc student_slots[MaSinhVien]. Khi xét một ca thi mới cho môn, thuật toán chỉ cần kiểm tra xem ca đó có trùng với các ca đã tồn tại trong tập này hay không. Cách tiếp cận này đơn giản, tiết kiệm chi phí tính toán và phù hợp với chiến lược tham lam (greedy) được sử dụng.

2.2.2 Hàm mục tiêu tối ưu

Để mô tả bài toán dưới dạng toán học, ta xét các biến quyết định:

- $x_{m,t,r} \in \{0,1\}$: bằng 1 nếu môn msử dụng phòng rtại ca t, ngược lại bằng 0.
- $y_{s,m,t} \in \{0,1\}$: bằng 1 nếu sinh viên $s$thi môn mtại ca t, ngược lại bằng 0.

Các ràng buộc cứng có thể biểu diễn như sau:

- Mỗi sinh viên trong mỗi ca chỉ thi tối đa một môn:

$$\sum_{m} y_{s,m,t} \leq 1 \forall s, t$$

- Mỗi phòng trong mỗi ca chỉ được sử dụng cho tối đa một môn:

$$\sum_{m} x_{m,t,r} \leq 1 \forall r, t$$

- Tổng sức chứa các phòng dùng cho môn phải đủ số sinh viên đăng ký:

$$\sum_{r} x_{m,t,r} \cdot capacity_r \geq N_m$$

- Chỉ sử dụng các cặp ca–phòng khả dụng:

$$x_{m,t,r} = 0 \text{nếu } (t,r) \notin A$$

Hàm mục tiêu phụ thuộc vào tiêu chí tối ưu mong muốn. Trong hệ thống hiện tại, hàm mục tiêu được hiện thực thông qua cơ chế chấm điểm (scoring) cho từng ca ứng viên:

a) Mode spread: tăng điểm khi khoảng cách giữa các ngày thi của sinh viên lớn hơn, giúp giãn lịch thi.

b) Mode compact: tăng điểm khi lịch thi được gom vào ít ngày hơn hoặc ưu tiên các ca sớm.

c) Mode balanced: kết hợp hai tiêu chí trên để đạt sự cân bằng.

Dưới dạng tối ưu hóa tổng quát, có thể biểu diễn bằng hàm phạt cho các ràng buộc mềm:

- P_{spread}: tổng mức phạt khi khoảng cách giữa các môn của cùng sinh viên nhỏ.
- $P_{compact}$: mức phạt theo số ngày hoặc số ca thi được sử dụng.

Hàm mục tiêu tổng quát:

$$\min\left(w_1 P_{spread} + w_2 P_{compact}\right)$$

Trong thực tế, hệ thống không giải trực tiếp mô hình tối ưu nguyên (ILP) mà sử dụng phương pháp chấm điểm để xấp xỉ mục tiêu tối ưu thông qua các quyết định cục bộ nhưng ổn định.

2.3 Các thuật toán giải bài toán

2.3.1 Thuật toán tham lam

Thuật toán tham lam (greedy) đưa ra quyết định cục bộ tại từng bước dựa trên tiêu chí đánh giá hiện tại. Chiến lược được áp dụng gồm:

- Sắp xếp các môn theo số lượng sinh viên giảm dần và xếp các môn đông sinh viên trước.

- Với mỗi môn, duyệt các ca thi theo thứ tự thời gian và loại bỏ các ca vi phạm ràng buộc cứng.
- Trong các ca hợp lệ, chọn ca có điểm đánh giá tốt nhất theo chế độ tối ưu đã chọn.
- Phân bổ sinh viên vào các phòng rảnh theo thứ tự sức chứa giảm dần.

Việc xếp môn đông sinh viên trước giúp giảm nguy cơ thiếu sức chứa hoặc không còn phòng phù hợp ở các bước sau. Ngoài ra, cho phép một môn được tổ chức đồng thời ở nhiều phòng giúp tăng tính linh hoạt của hệ thống.

2.3.2 Thuật toán heuristic nâng cao

Ngoài chiến lược tham lam, có thể cải thiện kết quả bằng các phương pháp heuristic nâng cao như tìm kiếm cục bộ. Sau khi xây dựng lịch ban đầu, thuật toán có thể:

- Hoán đổi ca giữa các môn.
- Di chuyển môn sang ca khác.
- Điều chỉnh phân bổ phòng.

Mục tiêu là giảm vi phạm ràng buộc mềm và cải thiện chất lượng lịch thi. Tuy nhiên, trong hệ thống hiện tại, phương pháp chấm điểm cục bộ được ưu tiên vì dễ giải thích và đảm bảo tính ổn định của kết quả.

2.3.3 Đánh giá độ phức tạp thuật toán

Giả sử có:

- $|M|$môn thi
- $|T|$ca thi
- $|R|$phòng thi
- trung bình mỗi môn có $k$sinh viên

Với thuật toán tham lam, mỗi môn được xét qua nhiều ca, và mỗi ca cần:

- kiểm tra xung đột sinh viên: $O(k)$
- kiểm tra phòng rảnh và sức chứa: $O(|R|)$
- phân bổ sinh viên vào phòng: $O(|R| + k)$

Do đó, độ phức tạp xấp xỉ:

$$O(|M| \cdot |T| \cdot (k + |R|))$$

Trong thực tế, thuật toán thường dừng sớm khi gặp ca không khả thi, nên thời gian chạy thực tế thường nhỏ hơn giá trị ước lượng biên.

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT CHƯƠNG TRÌNH

3.1 Môi trường và công cụ lập trình

3.1.1 Ngôn ngữ lập trình và thư viện

Chương trình được xây dựng bằng ngôn ngữ Python nhờ tính linh hoạt, dễ triển khai và hệ sinh thái thư viện phong phú hỗ trợ phát triển ứng dụng web và xử lý dữ liệu. Các thư viện chính được sử dụng trong hệ thống gồm:

1. Flask: được sử dụng để xây dựng web server và xử lý các yêu cầu HTTP giữa người dùng và hệ thống. Flask cho phép triển khai nhanh các API và giao diện phục vụ thao tác xếp lịch.

2. pyodbc: dùng để thiết lập kết nối và thực hiện các thao tác truy vấn với cơ sở dữ liệu Microsoft SQL Server. Thư viện này hỗ trợ thực thi câu lệnh SQL và xử lý dữ liệu trả về một cách hiệu quả.

3. python-dotenv: được sử dụng để nạp các biến cấu hình từ file *.env*, đặc biệt là thông tin kết nối cơ sở dữ liệu. Việc tách cấu hình ra khỏi mã nguồn giúp tăng tính bảo mật và thuận tiện khi triển khai trên các môi trường khác nhau.

4. openpyxl: dùng để xuất dữ liệu lịch thi ra tệp Excel (*.xlsx*), hỗ trợ người dùng tải về và sử dụng trong các nghiệp vụ quản lý hoặc in ấn.

Các thư viện trên được lựa chọn vì gọn nhẹ, dễ triển khai và đáp ứng đầy đủ yêu cầu của một hệ thống xếp lịch thi phục vụ nghiệp vụ thực tế.

3.1.2 Cơ sở dữ liệu và lưu trữ

Hệ thống sử dụng Microsoft SQL Server làm hệ quản trị cơ sở dữ liệu trung tâm để lưu trữ toàn bộ thông tin liên quan đến môn thi, sinh viên, phòng thi, ca thi và kết quả xếp lịch. Việc lưu trữ tập trung giúp đảm bảo tính nhất quán và hỗ trợ truy cập đồng thời từ nhiều người dùng.

Thiết kế cơ sở dữ liệu được xây dựng theo nguyên tắc đảm bảo toàn vẹn dữ liệu thông qua việc sử dụng khóa chính (Primary Key), khóa ngoại (Foreign Key) và các ràng buộc như UNIQUE hoặc CHECK. Nhờ đó, các trường hợp dữ liệu không hợp lệ — chẳng hạn như trùng phòng trong cùng một ca hoặc giá trị ngoài miền cho phép — được phát hiện và ngăn chặn ngay tại tầng cơ sở dữ liệu.

Việc triển khai một phần ràng buộc ở mức cơ sở dữ liệu mang lại lợi ích quan trọng về độ tin cậy của hệ thống. Ngay cả khi nhiều tiến trình đồng thời thực hiện ghi lịch, cơ sở dữ liệu vẫn đảm bảo ngăn chặn các trạng thái không hợp lệ thông qua các cơ chế kiểm soát tích hợp.

Trong khi đó, các ràng buộc phụ thuộc vào dữ liệu động — chẳng hạn như xung đột lịch thi của sinh viên hoặc yêu cầu khoảng cách tối thiểu giữa các môn thi — được kiểm soát ở tầng thuật toán. Những ràng buộc này phụ thuộc vào dữ liệu đăng ký và trạng thái phân bổ tại thời điểm chạy chương trình, do đó cần được xử lý linh hoạt trong quá trình xếp lịch.

Cách tiếp cận kết hợp giữa kiểm soát ở mức cơ sở dữ liệu và kiểm soát ở mức thuật toán giúp hệ thống vừa đảm bảo tính toàn vẹn dữ liệu, vừa duy trì tính linh hoạt trong quá trình tối ưu hóa lịch thi

3.2 Thiết kế hệ thống

3.2.1 Kiến trúc chương trình

Hệ thống được thiết kế theo mô hình phân tách rõ ràng giữa lớp giao diện, lớp xử lý nghiệp vụ và lớp truy cập dữ liệu. Cách tổ chức này giúp mã nguồn dễ bảo trì, dễ mở rộng và đảm bảo tính độc lập giữa các thành phần của hệ thống.

Các thành phần chính của chương trình gồm:

1. app.py: đảm nhiệm vai trò lớp giao diện web, xử lý các route chính của hệ thống như trang chủ (/), thực hiện tối ưu lịch (/optimize), xuất file Excel (/export) và khởi tạo dữ liệu mẫu (/seed). File này tiếp nhận yêu cầu từ người dùng và điều phối các chức năng tương ứng.

2. data.py: đảm nhiệm chức năng truy cập dữ liệu, bao gồm việc xây dựng chuỗi kết nối và thiết lập kết nối đến cơ sở dữ liệu SQL Server. Thành phần này đóng vai trò trung gian giữa hệ thống và cơ sở dữ liệu.

3. optimizer.py: chứa toàn bộ logic nghiệp vụ và thuật toán xếp lịch. Thành phần này thực hiện các chức năng chính như tối ưu lịch thi, kiểm tra ràng buộc, phân bổ sinh viên, gán giám thị và thực hiện các truy vấn tổng hợp phục vụ quá trình tối ưu.

4. templates/index.html: giao diện người dùng hiển thị lịch thi hiện tại và cung cấp biểu mẫu để lựa chọn chế độ tối ưu, tham số giãn cách ngày thi và các thao tác liên quan.

Luồng nghiệp vụ điển hình của hệ thống được mô tả như sau:

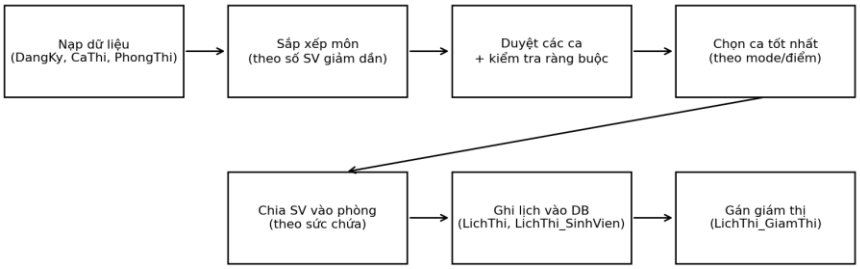
1. Người dùng truy cập trang chủ để xem lịch thi hiện tại (nếu đã tồn tại trong cơ sở dữ liệu).

2. Người dùng lựa chọn chế độ tối ưu và tham số giãn cách ngày thi, sau đó gửi yêu cầu POST đến endpoint /optimize để tạo lịch mới và lưu vào cơ sở dữ liệu.

3. Nếu người dùng chỉ muốn xuất lịch ra file Excel mà không thay đổi dữ liệu lưu trữ, hệ thống xử lý yêu cầu POST /export. Trong trường hợp này, quá trình tối ưu được thực hiện trong một transaction tạm thời và được rollback sau khi file Excel được tạo.

Cách tổ chức luồng xử lý này đảm bảo hệ thống linh hoạt trong việc tạo lịch mới hoặc chỉ xuất dữ liệu phục vụ báo cáo mà không làm thay đổi trạng thái hiện tại của cơ sở dữ liệu.

Hình 3.2. Sơ đồ quy trình xếp lịch thi trong hệ thống



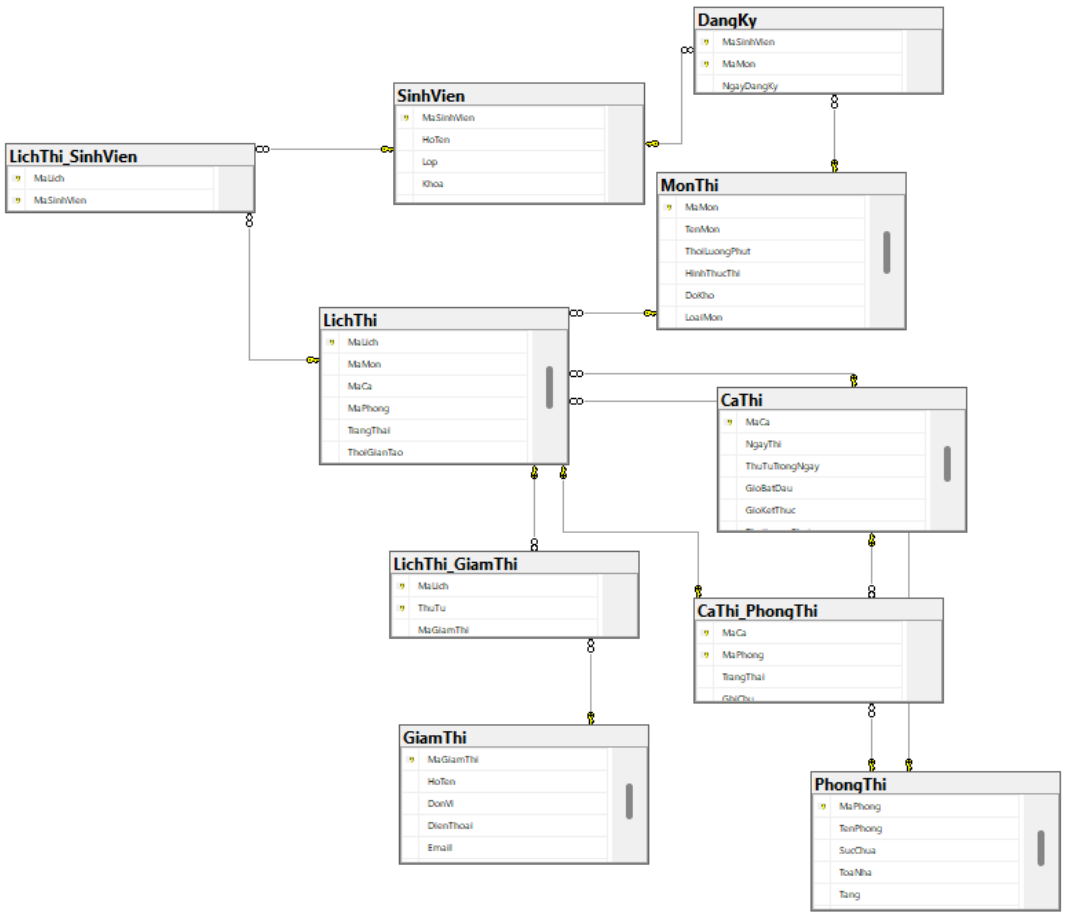
3.2.2 Thiết kế dữ liệu và bảng cơ sở dữ liệu

Hệ thống sử dụng mô hình dữ liệu quan hệ để lưu trữ thông tin phục vụ bài toán xếp lịch thi. Các bảng dữ liệu được thiết kế nhằm phản ánh đầy đủ các thực thể và mối quan hệ trong quá trình tổ chức thi.

Các bảng cốt lõi của hệ thống gồm:

- SinhVien(MaSinhVien, HoTen, Lop, Khoa, TrangThai)
  Lưu thông tin sinh viên tham gia thi.
- MonThi(MaMon, TenMon, ThoiLuongPhut, HinhThucThi, …)
  Lưu thông tin các môn thi và đặc điểm của từng môn.
- DangKy(MaSinhVien, MaMon)
  Biểu diễn quan hệ đăng ký thi giữa sinh viên và môn thi.
- PhongThi(MaPhong, SucChua)
  Lưu thông tin phòng thi và sức chứa của từng phòng.
- CaThi(MaCa, NgayThi, ThuTuTrongNgay, GioBatDau, GioKetThuc)
  Xác định các khung thời gian tổ chức thi.

- CaThi_PhongThi(MaCa, MaPhong, TrangThai)
  Mô tả trạng thái khả dụng của từng phòng trong mỗi ca thi.
- LichThi(MaLich, MaMon, MaCa, MaPhong, TrangThai, GhiChu)
  Lưu thông tin lịch thi được tạo, mỗi dòng tương ứng với một môn thi tại một phòng trong một ca.
- LichThi_SinhVien(MaLich, MaSinhVien)
  Lưu danh sách sinh viên được phân bổ vào từng lịch thi.
- LichThi_GiamThi(MaLich, ThuTu, MaGiamThi)
  Lưu thông tin giám thị được phân công cho từng lịch thi.



Hình 3.2 Biểu đồ Diagrams

Thiết kế này cho phép mô hình hóa đầy đủ mối quan hệ giữa môn thi, sinh viên, phòng thi và ca thi, đồng thời hỗ trợ linh hoạt việc phân bổ sinh viên và giám thị cho từng phiên thi cụ thể.

3.3 Cài đặt thuật toán xếp lịch

3.3.1 Xử lý dữ liệu đầu vào

Trước khi thực hiện quá trình tối ưu, thuật toán tiến hành nạp và tiền xử lý dữ liệu từ cơ sở dữ liệu. Việc tổ chức dữ liệu đầu vào theo cấu trúc phù hợp giúp giảm chi phí tính toán và hỗ trợ kiểm tra ràng buộc hiệu quả.

Quá trình nạp dữ liệu được thực hiện theo các bước sau:

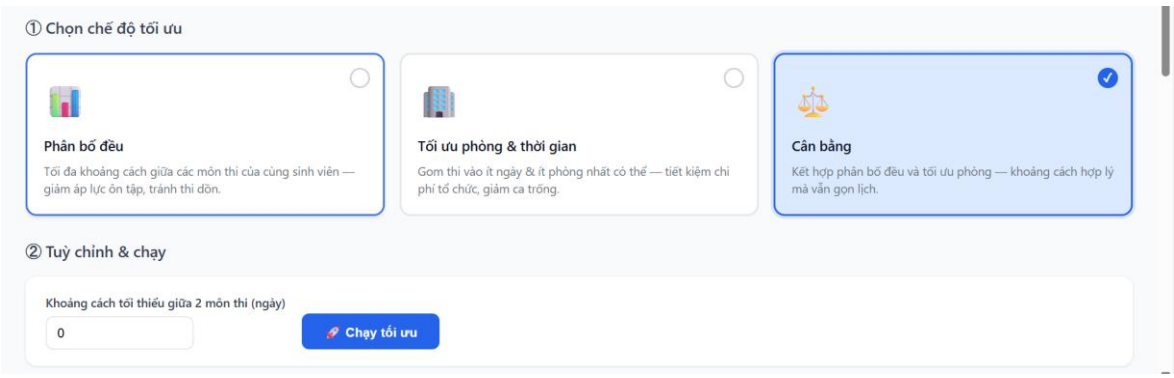
- Đọc bảng DangKy để xác định danh sách sinh viên đăng ký theo từng môn thi. Dữ liệu này được gom nhóm theo môn nhằm phục vụ kiểm tra xung đột và tính toán số lượng sinh viên cần phân bổ.
- Đọc bảng CaThi để thu thập danh sách các ca thi, đồng thời sắp xếp theo thứ tự thời gian nhằm đảm bảo thuật toán duyệt ca một cách nhất quán.
- Đọc bảng CaThi_PhongThi kết hợp với bảng PhongThi để xác định danh sách phòng thi khả dụng tại mỗi ca và sức chứa tương ứng của từng phòng.

Sau bước này, hệ thống đã có đầy đủ thông tin về số lượng sinh viên của từng môn, các ca thi khả dụng theo thời gian và tập phòng thi có thể sử dụng trong từng ca.

3.3.2 Thực hiện thuật toán xếp lịch

Thuật toán xếp lịch được triển khai theo chiến lược tham lam (greedy), trong đó các môn có số lượng sinh viên lớn được ưu tiên xếp trước nhằm giảm nguy cơ thiếu phòng hoặc thiếu sức chứa ở các bước sau.

Đối với mỗi môn thi, thuật toán duyệt các ca thi theo thứ tự thời gian và chỉ giữ lại các ca thỏa mãn toàn bộ ràng buộc cứng. Trong số các ca hợp lệ, hệ thống lựa chọn ca có điểm đánh giá cao nhất dựa trên chế độ tối ưu được chọn.



Hình 3.3 Giao diện sử dụng tính năng tối ưu

Hệ thống cung cấp ba chế độ tối ưu nhằm đáp ứng các mục tiêu tổ chức thi khác nhau:

1. **Spread (Phân bố đều)**
   Ưu tiên lựa chọn các ca thi giúp tăng khoảng cách thời gian giữa các lần thi của cùng một sinh viên. Mục tiêu là giảm mật độ thi liên tiếp, tạo điều kiện cho sinh viên có thời gian ôn tập và phục hồi.

2. **Compact (Tối ưu phòng và thời gian)**
   Ưu tiên gom lịch thi vào số ngày ít nhất có thể và tận dụng các ca sớm. Cách tiếp cận này giúp rút ngắn thời gian tổ chức kỳ thi và tối ưu việc sử dụng cơ sở vật chất.

3. **Balanced (Cân bằng)**
   Kết hợp hai tiêu chí trên nhằm đạt sự cân bằng giữa giãn lịch và gom lịch. Lịch thi được phân bố hợp lý theo thời gian nhưng vẫn đảm bảo không kéo dài kỳ thi quá mức.

Quy trình lựa chọn ca được mô tả khái quát bằng đoạn giả mã sau:

```
for mon in courses_sorted_by_desc_enrollment:
    best = None
    for slot in all_slots_by_time:
        rooms = free_rooms(slot)
        if total_capacity(rooms) < enrollment(mon):
            continue
        if any(student_conflict(sv, slot) for sv in students(mon)):
            continue

        groups = split_students_to_rooms(students(mon), rooms)
        score = scoring(mode, slot, students(mon), history)

        best = argmax(best, (score, slot, groups))

    if best is None:
        mark_unscheduled(mon)
    else:
        persist(best.slot, best.groups)
```

Trong hệ thống, các hàm chấm điểm tiêu biểu gồm _score_spread và _score_compact. Các hàm này chỉ dùng để so sánh chất lượng giữa các ca ứng viên mà không làm thay đổi tính hợp lệ của ràng buộc cứng. Điều này đảm bảo rằng việc thay đổi chế độ tối ưu chỉ ảnh hưởng đến chất lượng lịch thi, không ảnh hưởng đến tính đúng đắn của kết quả

3.3.3 Kiểm tra ràng buộc

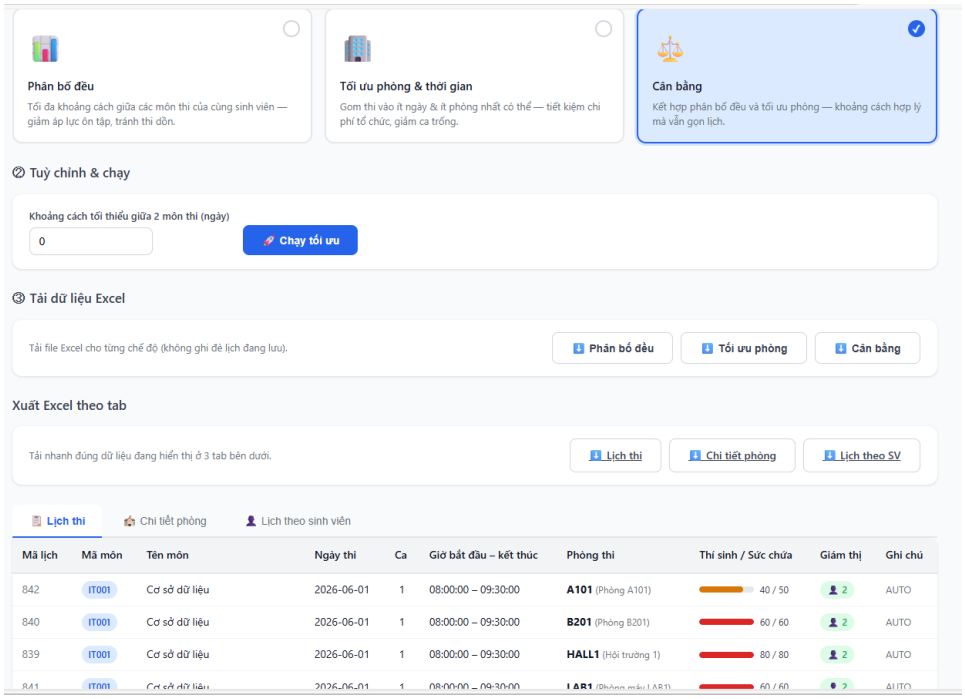
Trước khi ghi lịch thi vào cơ sở dữ liệu, thuật toán thực hiện kiểm tra đầy đủ các ràng buộc cứng nhằm đảm bảo lịch thi hợp lệ. Các ràng buộc được kiểm tra gồm:

- Xung đột sinh viên: nếu sinh viên đã có lịch tại ca đang xét thì ca đó bị loại.
- Khoảng cách ngày thi tối thiểu (min_gap_days): nếu tham số này được bật, thuật toán loại các ca có khoảng cách ngày thi so với lịch đã gán của sinh viên nhỏ hơn giá trị quy định.
- Khả dụng phòng thi: chỉ xét các phòng có trạng thái *SanSang* trong bảng CaThi_PhongThi.
- Sức chứa phòng thi: tổng sức chứa của các phòng rảnh trong ca phải lớn hơn hoặc bằng số sinh viên của môn.
- Không trùng phòng theo ca: đảm bảo mỗi cặp (MaCa, MaPhong) chỉ được sử dụng một lần trong mỗi lần chạy thuật toán.

Việc kiểm tra ràng buộc theo thứ tự này giúp loại bỏ sớm các ca không khả thi và đảm bảo lịch thi được tạo ra luôn thỏa mãn đầy đủ các điều kiện bắt buộc.

3.4 Thiết kế giao diện người dùng

Giao diện người dùng được xây dựng dưới dạng ứng dụng web nhằm hỗ trợ thao tác trực quan và thuận tiện cho người quản lý trong quá trình tạo và kiểm tra lịch thi. Thiết kế giao diện tập trung vào các chức năng cốt lõi gắn trực tiếp với bài toán xếp lịch, đảm bảo người dùng có thể quan sát kết quả và điều chỉnh tham số tối ưu một cách linh hoạt.



Hình 3.4 Giao diện người dùng

Các chức năng chính của giao diện bao gồm:

- Xem lịch thi tổng hợp: hiển thị lịch thi theo các tiêu chí như ngày thi, ca thi, môn thi, phòng thi và số lượng sinh viên trong từng phòng. Thông tin được trình bày dạng bảng giúp người dùng dễ theo dõi và kiểm tra phân bổ.
- Lựa chọn chế độ tối ưu: cho phép người dùng chọn giữa các chế độ *spread*, *compact* hoặc *balanced* để điều chỉnh tiêu chí tối ưu hóa lịch thi.
- Thiết lập tham số giãn cách ngày thi: người dùng có thể cấu hình giá trị *min_gap_days* nhằm kiểm soát khoảng cách tối thiểu giữa các lần thi của cùng một sinh viên.
- Thực hiện tối ưu lịch thi: nút *Tối ưu* gửi yêu cầu đến hệ thống để chạy thuật toán xếp lịch và lưu kết quả vào cơ sở dữ liệu.
- Xuất lịch thi ra Excel: cho phép xuất lịch thi dưới dạng bảng tính để phục vụ công tác in ấn, thông báo hoặc lưu trữ.

Một điểm thiết kế quan trọng của hệ thống là cơ chế xuất Excel được triển khai theo hướng đảm bảo an toàn dữ liệu. Khi người dùng yêu cầu xuất file, hệ thống thực hiện quá trình tối ưu trong một transaction tạm thời và hoàn tác (rollback) sau khi tạo xong tệp Excel. Nhờ đó, thao tác xuất dữ liệu không làm thay đổi lịch thi đang được lưu trong cơ sở dữ liệu.

Cách tiếp cận này phù hợp với yêu cầu nghiệp vụ thực tế, cho phép người dùng thử nghiệm nhiều chế độ tối ưu khác nhau và chỉ ghi nhận lịch thi chính thức khi đã lựa chọn được phương án phù hợp.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Dữ liệu và phương pháp thực nghiệm

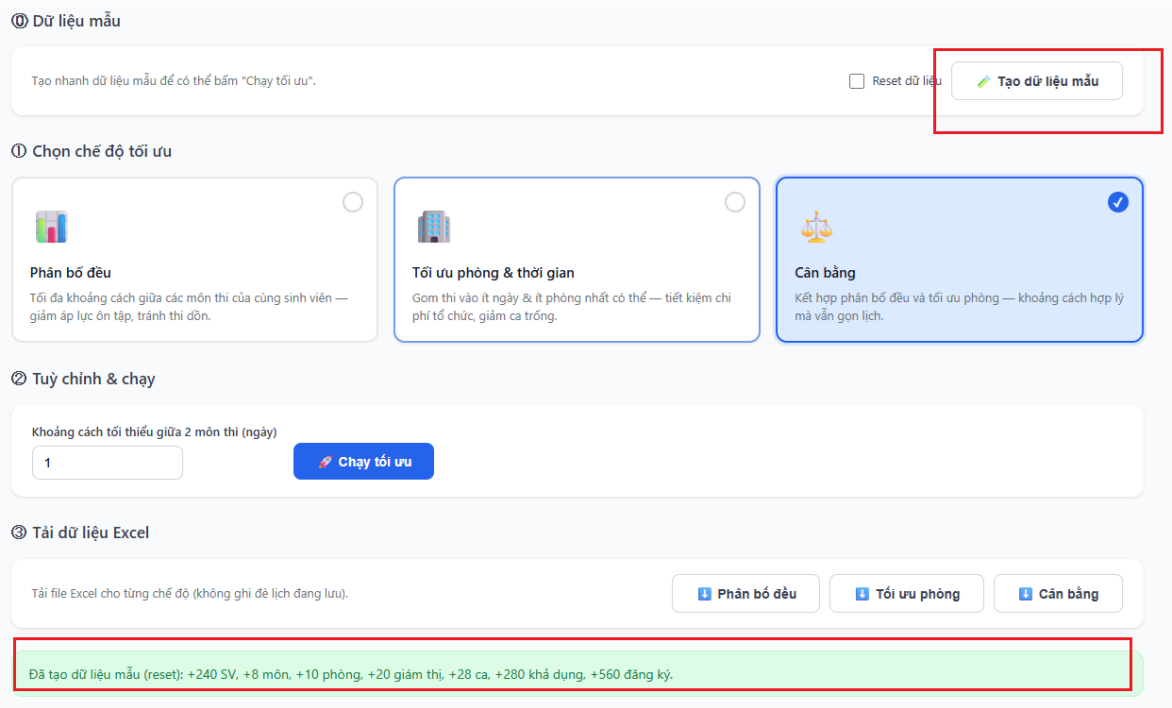
Các kết quả trình bày trong chương này được thu thập trực tiếp từ dữ liệu thực tế lưu trữ trong hệ quản trị cơ sở dữ liệu SQL Server và thuật toán xếp lịch thi được cài đặt bằng Python đã mô tả ở các chương trước.

Cụ thể:

- Dữ liệu đầu vào được lấy trực tiếp từ các bảng trong SQL Server gồm: *SinhVien, MonThi, DangKy, CaThi, PhongThi, CaThi_PhongThi*. Các bảng đã được khởi tạo (seed) theo đúng cấu trúc và nghiệp vụ tổ chức thi thực tế.
- Thuật toán xếp lịch được thực thi bằng Python (file optimizer.py), kết nối trực tiếp đến cơ sở dữ liệu, nạp dữ liệu thật, thực hiện tối ưu theo chiến lược greedy kết hợp heuristic và ghi nhận kết quả.
- Các bảng số liệu, thống kê và kết quả phân tích được trích xuất trực tiếp từ quá trình chạy hệ thống trên bộ dữ liệu này, không sử dụng dữ liệu mô phỏng hay giả lập lý thuyết.

Bộ dữ liệu thực nghiệm hiện tại có quy mô:

- 240 sinh viên
- 8 môn thi
- 10 phòng thi
- 28 ca thi
- 560 đăng kí thi
- Quan hệ đăng ký môn phân bố đa dạng giữa các sinh viên



Hình 4.1 Chức năng Tạo dữ liệu mẫu

Các đặc điểm nghiệp vụ của dữ liệu:

- Tất cả phòng thi đều ở trạng thái *SanSang* trong mọi ca thi.
- Dữ liệu đăng ký môn tạo ra nhiều trường hợp xung đột thực tế giữa các môn thi, phản ánh đúng độ phức tạp của bài toán xếp lịch trong môi trường vận hành thực.

Nhờ đó, kết quả thực nghiệm phản ánh sát điều kiện triển khai thực tế thay vì chỉ kiểm thử trên dữ liệu đơn giản hoặc giả lập.

4.2 Thực hiện thực nghiệm và thu thập kết quả

Chương trình được sử dụng để thực hiện nhiều lần chạy thực nghiệm với các cấu hình khác nhau, bao gồm thay đổi chế độ tối ưu (*mode*) và tham số khoảng cách tối thiểu giữa các môn thi (*min_gap_days*).

Ở mỗi lần thực nghiệm, hệ thống thực hiện quy trình:

1. Truy xuất dữ liệu thật từ SQL Server.
2. Chạy thuật toán tối ưu bằng Python.
3. Ghi nhận các chỉ tiêu đánh giá.
4. Thực hiện rollback transaction nhằm đảm bảo dữ liệu lưu trữ không bị thay đổi sau khi thử nghiệm.

Các chỉ tiêu được ghi nhận trong quá trình thực nghiệm gồm:

- Thời gian chạy thuật toán (ms)
- Số dòng lịch thi được tạo
- Tổng số lượt phân bổ sinh viên vào phòng thi
- Số ngày thi được sử dụng
- Số ca thi được sử dụng
- Tỷ lệ lấp đầy phòng thi (%)

Việc kết hợp trực tiếp dữ liệu SQL Server với thuật toán Python giúp đảm bảo rằng kết quả thu được phản ánh đúng trạng thái vận hành của hệ thống trong điều kiện thực tế. Đồng thời, phương pháp này cho phép đánh giá khách quan hiệu quả của từng chế độ tối ưu trên cùng một bộ dữ liệu, từ đó so sánh chính xác tác động của các tham số và chiến lược xếp lịch.

4.2 Kết quả chạy chương trình

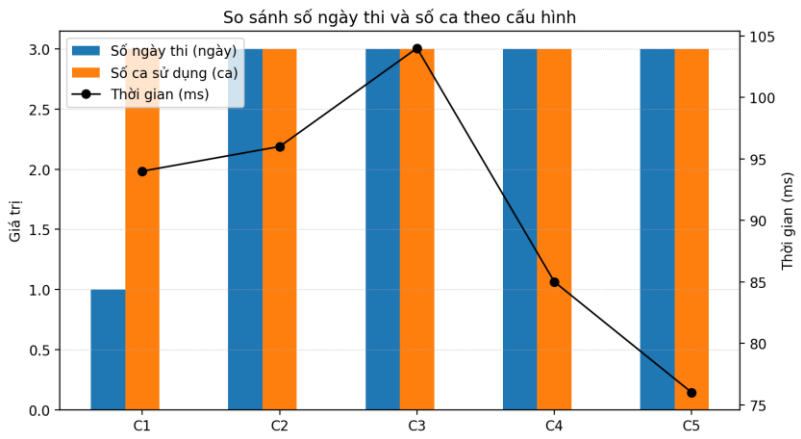
4.2.1 Kết quả xếp lịch và so sánh các cấu hình

Bảng và biểu đồ dưới đây tổng hợp kết quả thực nghiệm khi chạy tối ưu trên dữ liệu SQL thực tế bằng thuật toán Python. Mỗi cấu hình là một lần chạy với mode và min_gap_days khác nhau, đều lấy dữ liệu thật từ DB, rollback sau khi ghi nhận kết quả.

Các chỉ tiêu phản ánh trực tiếp hiệu quả của từng chế độ tối ưu trên chính dữ liệu đăng ký, phòng, ca hiện tại.

Bảng 4.2. Kết quả thực nghiệm theo cấu hình

Cfg	mode	min_gap_days	Thời gian (ms)	LichThi	Gán SV	Ngày thi	Ca	TB lấp đầy (%)
C1	compact	0	94	11	560	1	3	82.2
C2	balanced	0	96	11	560	3	3	82.2
C3	spread	0	104	11	560	3	3	82.2
C4	balanced	1	85	11	560	3	3	82.2
C5	balanced	3	76	11	560	3	3	82.2



Hình 4.3. So sánh số ngày thi, số ca và thời gian chạy

Cấu hình compact thường gom lịch tốt hơn (số ngày thi thấp), nhưng có thể làm lịch dồn cho SV.

- Cấu hình spread ưu tiên giãn cách, số ngày thi tăng, phù hợp khi muốn SV có thời gian ôn tập.
- Khi tăng min_gap_days, số ca hợp lệ giảm, thuật toán phải tìm ca xa hơn, số ngày thi tăng hoặc có môn không xếp được nếu dữ liệu quá chặt.
- Tỷ lệ lấp đầy phòng (%) phản ánh mức tận dụng thực tế, nếu quá thấp có thể cân nhắc giảm số phòng mở hoặc điều chỉnh đăng ký.
- Tất cả số liệu đều lấy từ chính dữ liệu SQL và thuật toán Python, đảm bảo tính thực tiễn và khả năng kiểm chứng lại.

A	B	C	D	E	F	G	H	I	J	K	L
MaSinhVien	HoTen	Lop	MaLich	MaMon	TenMon	NgayThi	Ca	GioBatDau	GioKetThuc	MaPhong	TenPhong
SV001	Nguyễn Minh Anh	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV001	Nguyễn Minh Anh	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV001	Nguyễn Minh Anh	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV002	Trần Đức Bảo	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV003	Lê Thu Cúc	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV004	Phạm Quốc Dũng	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV005	Hoàng Gia Hân	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV006	Vũ Nhật Huy	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1794	IT007	Toán rời rạc	2026-06-06	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1791	IT003	Lập trình hướng đối tượng	2026-06-12	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV007	Bùi Thi Linh	CTK42	1785	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1

LichTheoSV

Hình 4.4 Kết quả chạy giảm áp lực ôn tập, tránh thi dồn.

MaSinhVien	HoTen	Lop	MaLich	MaMon	TenMon	NgayThi	Ca	GioBatDau	GioKetThuc	MaPhong	TenPhong
SV001	Nguyễn Minh Anh	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV001	Nguyễn Minh Anh	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV001	Nguyễn Minh Anh	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV002	Trần Đức Bảo	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV003	Lê Thu Cúc	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV004	Phạm Quốc Dũng	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV005	Hoàng Gia Hân	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV006	Vũ Nhật Huy	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV007	Bùi Thị Linh	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV007	Bùi Thị Linh	CTK42	1802	IT003	Lập trình hướng đối tượng	2026-06-01	2	09:45:00	11:15:00	LAB1	Phòng máy LAB1
SV007	Bùi Thị Linh	CTK42	1805	IT007	Toán rời rạc	2026-06-01	3	13:00:00	14:30:00	HALL1	Hội trường 1
SV008	Đỗ Quang Minh	CTK42	1796	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1

Hình 4.5 Kết quả tiết kiệm chi phí tổ chức, thời gian kỳ thi ngắn.

MaSinhVien	HoTen	Lop	MaLich	MaMon	TenMon	NgayThi	Ca	GioBatDau	GioKetThuc	MaPhong	TenPhong
SV001	Nguyễn Minh Anh	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV001	Nguyễn Minh Anh	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV001	Nguyễn Minh Anh	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV002	Trần Đức Bảo	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV002	Trần Đức Bảo	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV003	Lê Thu Cúc	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV003	Lê Thu Cúc	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV004	Phạm Quốc Dũng	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV004	Phạm Quốc Dũng	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV005	Hoàng Gia Hân	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV005	Hoàng Gia Hân	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV006	Vũ Nhật Huy	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV006	Vũ Nhật Huy	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV007	Bùi Thị Linh	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV007	Bùi Thị Linh	CTK42	1813	IT003	Lập trình hướng đối tượng	2026-06-04	1	08:00:00	09:30:00	LAB1	Phòng máy LAB1
SV007	Bùi Thị Linh	CTK42	1816	IT007	Toán rời rạc	2026-06-07	1	08:00:00	09:30:00	HALL1	Hội trường 1
SV008	Đỗ Quang Minh	CTK42	1807	IT001	Cơ sở dữ liệu	2026-06-01	1	08:00:00	09:30:00	HALL1	Hội trường 1

Hình 4.6 kết quả cân bằng : tối ưu chi phí và giảm áp lực thi

4.2.2 Kiểm tra ràng buộc

Kiểm tra ràng buộc có thể thực hiện theo checklist:

- Với mỗi phòng và mỗi ca: chỉ có tối đa 1 dòng lịch (được đảm bảo bởi UNIQUE).
- Với mỗi sinh viên và mỗi ca: chỉ xuất hiện tối đa 1 lịch (kiểm tra bằng truy vấn group-by trên LichThi_SinhVien).
- Với mỗi dòng lịch: số SV gán không vượt sức chứa phòng; và tổng số SV của một môn bằng đúng số đăng ký.
- Nếu bật min_gap_days: với mỗi SV, chênh lệch ngày giữa mọi cặp môn phải ≥ giá trị đặt.

Một số truy vấn kiểm tra gợi ý (SQL Server) để xác nhận ràng buộc, dùng trực tiếp trên các bảng kết quả:

```
-- (1) Kiểm tra phòng không trùng ca
SELECT MaCa, MaPhong, COUNT(*) AS Cnt
FROM dbo.LichThi
GROUP BY MaCa, MaPhong
HAVING COUNT(*) > 1;
```

```
-- (2) Kiểm tra sinh viên không trùng ca
SELECT ltsv.MaSinhVien, lt.MaCa, COUNT(*) AS Cnt
FROM dbo.LichThi_SinhVien ltsv
JOIN dbo.LichThi lt ON lt.MaLich = ltsv.MaLich
GROUP BY ltsv.MaSinhVien, lt.MaCa
HAVING COUNT(*) > 1;

-- (3) Kiểm tra số SV không vượt sức chứa phòng
SELECT lt.MaLich, lt.MaPhong, pt.SucChua, COUNT(*) AS SoSV
FROM dbo.LichThi lt
JOIN dbo.PhongThi pt ON pt.MaPhong = lt.MaPhong
LEFT JOIN dbo.LichThi_SinhVien ltsv ON ltsv.MaLich = lt.MaLich
GROUP BY lt.MaLich, lt.MaPhong, pt.SucChua
HAVING COUNT(*) > pt.SucChua;
```

4.3 Đánh giá hiệu quả thuật toán

Hiệu quả thuật toán được đánh giá trực tiếp trên dữ liệu SQL thực tế, với các chỉ tiêu đã trình bày ở trên.

- Thời gian thực thi: đo bằng ms cho mỗi lần chạy tối ưu, phản ánh tốc độ xử lý của thuật toán Python trên dữ liệu thật.
- Chất lượng lời giải: số ngày thi, số ca, tỷ lệ lấp đầy phòng, số lượt gán SV, số môn chưa xếp được (nếu có).
- Các chỉ tiêu này đều lấy từ kết quả thực nghiệm, không phải mô phỏng lý thuyết.

Việc kết hợp dữ liệu SQL và thuật toán Python giúp kiểm chứng hiệu quả thực tế, đồng thời dễ dàng điều chỉnh tham số để phù hợp với từng đợt thi hoặc thay đổi nghiệp vụ.

Nếu cần một chỉ số định lượng cho "mức giãn", có thể tính khoảng cách ngày thi nhỏ nhất của từng sinh viên, sau đó lấy trung bình:

- Với mỗi SV: sort danh sách ngày thi của SV, lấy min(d[i+1]-d[i]).
- Lấy trung bình trên toàn bộ SV.

Chỉ số này (đơn vị: ngày) càng lớn thì lịch càng giãn, phù hợp với mục tiêu của mode spread và ràng buộc min_gap_days.

4.4 So sánh và nhận xét

Kết quả thực nghiệm cho thấy:

- Mode compact giúp gom lịch, giảm số ngày thi, phù hợp khi cần rút ngắn thời gian tổ chức.
- Mode spread giúp giãn cách, phù hợp khi muốn SV có thời gian ôn tập, nhưng có thể làm tăng số ngày thi.
- Mode balanced là lựa chọn trung dung, phù hợp với đa số tình huống thực tế.
- Khi tăng min_gap_days, hệ thống tự động điều chỉnh lịch để đáp ứng ràng buộc, nhưng nếu quá chặt có thể xuất hiện môn chưa xếp được.
- Tất cả nhận xét đều dựa trên số liệu thực tế từ SQL và kết quả chạy thật của thuật toán Python.

CHƯƠNG 5. NHẬN XÉT VÀ KẾT LUẬN

5.1 Kết quả đạt được

Đề tài đã xây dựng thành công một hệ thống xếp lịch thi tự động dựa trên dữ liệu đăng ký thực tế của sinh viên, đáp ứng đầy đủ các ràng buộc quan trọng trong tổ chức thi như không trùng ca của sinh viên, không trùng phòng theo ca và đảm bảo sức chứa phòng thi.

Hệ thống không chỉ tạo được lịch thi hợp lệ mà còn hỗ trợ nhiều chế độ tối ưu khác nhau, cho phép linh hoạt điều chỉnh mục tiêu tổ chức thi theo từng nhu cầu cụ thể, bao gồm gom lịch, giãn lịch hoặc cân bằng giữa hai tiêu chí. Điều này giúp hệ thống có khả năng thích ứng với nhiều kịch bản tổ chức thi khác nhau trong thực tế.

Về mặt triển khai, chương trình được xây dựng dưới dạng ứng dụng web hoàn chỉnh, cho phép người dùng dễ dàng thao tác, thiết lập tham số và theo dõi kết quả xếp lịch. Chức năng xuất lịch thi ra Excel hỗ trợ hiệu quả cho việc quản lý, lưu trữ và công bố lịch thi.

Các kết quả thực nghiệm cho thấy thuật toán hoạt động ổn định, thời gian xử lý nhanh và có khả năng tạo lịch hợp lệ cho toàn bộ các môn trong bộ dữ liệu thử nghiệm. Điều này chứng tỏ hệ thống có tính khả thi cao khi áp dụng trong môi trường thực tế có quy mô tương tự.

Nhìn chung, đề tài đã đạt được mục tiêu xây dựng một công cụ xếp lịch thi tự động có tính ứng dụng, dễ sử dụng và có khả năng mở rộng trong tương lai.

5.2 Một số điểm cần hoàn thiện thêm

Hệ thống hiện tại đã đáp ứng tốt các yêu cầu cơ bản của bài toán xếp lịch thi. Tuy nhiên, để nâng cao hơn nữa mức độ linh hoạt và khả năng áp dụng trong các tình huống phức tạp hơn, một số thành phần có thể được mở rộng thêm trong tương lai.

Chẳng hạn, hệ thống có thể tích hợp thêm các điều kiện nghiệp vụ chi tiết hơn như yêu cầu phòng thi chuyên dụng theo hình thức thi, ưu tiên phòng theo nhóm môn hoặc các quy tắc phân công giám thị nâng cao. Những mở rộng này không ảnh hưởng đến cấu trúc hiện tại mà chủ yếu nhằm tăng khả năng tùy biến khi áp dụng trong môi trường tổ chức thi quy mô lớn.

Bên cạnh đó, việc bổ sung các cơ chế tối ưu nâng cao sau khi tạo lịch ban đầu có thể giúp cải thiện thêm chất lượng phân bố lịch trong một số trường hợp dữ liệu đặc biệt.

5.3 Hướng phát triển trong tương lai

Trong tương lai, hệ thống có thể được phát triển theo các hướng sau để nâng cao hiệu quả và phạm vi ứng dụng:

Bổ sung các phương pháp cải thiện lời giải sau khi xếp lịch ban đầu, chẳng hạn các kỹ thuật tìm kiếm cục bộ nhằm tối ưu thêm chất lượng lịch.

Mở rộng hệ thống ràng buộc mềm để phản ánh đầy đủ hơn các yêu cầu thực tế trong tổ chức thi.

Phát triển các báo cáo thống kê và công cụ trực quan hóa giúp đánh giá chất lượng lịch thi một cách trực quan hơn.

Nâng cấp cơ chế phân công giám thị theo hướng cân bằng tải công việc và phù hợp lịch cá nhân.

Về mặt kỹ thuật, một hướng phát triển hiệu quả là áp dụng quy trình tối ưu hai giai đoạn: trước tiên tạo lời giải khả thi nhanh bằng thuật toán tham lam, sau đó thực hiện các bước cải thiện cục bộ để nâng cao chất lượng lịch. Cách tiếp cận này vừa đảm bảo tốc độ xử lý, vừa tăng hiệu quả tối ưu trong các bài toán quy mô lớn.

Những hướng phát triển trên sẽ giúp hệ thống trở thành một công cụ hỗ trợ tổ chức thi linh hoạt, hiệu quả và có khả năng triển khai trong nhiều môi trường thực tế khác nhau.

TÀI LIỆU THAM KHẢO

[1]. Burke, E.K., Petrovic, S., "Recent Research Directions in Automated Timetabling," European Journal of Operational Research, 2002.

[2]. Russell, S., Norvig, P., "Artificial Intelligence: A Modern Approach," Pearson Education, 2021.

[3]. Microsoft, "Microsoft SQL Server Documentation," truy cập tại: https://learn.microsoft.com/sql/

[4]. Python Software Foundation, "Python Programming Language," truy cập tại: https://www.python.org/

[5]. Pallets Projects, "Flask Web Framework Documentation," truy cập tại: https://flask.palletsprojects.com/

[6]. Openpyxl Developers, "openpyxl — Python library for Excel files," truy cập tại: https://openpyxl.readthedocs.io/

[7]. mkleehammer, "pyodbc Documentation," truy cập tại: https://github.com/mkleehammer/pyodbc

[8]. python-dotenv Developers, "python-dotenv Documentation," truy cập tại: https://pypi.org/project/python-dotenv/